

Project Documentation: Real-Time ISS Tracker

An end-to-end streaming architecture within Microsoft Fabric

This document contains the comprehensive step-by-step description and the complete source code used to build the real-time International Space Station (ISS) tracker within the Microsoft Fabric platform. This architecture captures, processes, cleans, and seamlessly visualizes live ISS coordinates on a NASA-style command center dashboard.

1. Architecture Overview

The streaming solution is modularly designed, leveraging the full capabilities of Real-Time Intelligence within Microsoft Fabric. The dataflow operates as follows:

Component	Role within the Architecture
Data Source	The public REST API from Open Notify (providing real-time satellite coordinates).
Fabric Environment	Centralized Spark environment containing the required external Python libraries.
Fabric Notebook	The active engine that polls the API every 5 seconds and pushes the results to the stream.
Data Pipeline	The orchestrator that runs the notebook as a permanent background stream (24/7).
Eventstream	The real-time data hub that captures and routes the incoming JSON payloads.
KQL Database	The ultra-fast storage layer within an Eventhouse, optimized for high-velocity streaming data.
Power BI Report	The live visualization layer featuring automatic map centering and Change Detection.

2. Step-by-Step Implementation

Step 1: Setting up the Fabric Environment

To stream data from a Fabric Notebook to an Eventstream, the official Azure Event Hub client library is required. To accommodate this, a dedicated **Environment** was created in the workspace.

- Under the **Public Libraries** section, the `azure-eventhub` module was added.
- Once published, this environment ensures that any attached Spark session automatically resolves code dependencies, completely avoiding `ModuleNotFoundError` exceptions.

Step 2: Data Pipeline & Automation

To ensure that the script runs independently and continuously without manual intervention, a Data Pipeline was configured (`pl_iss_tracker_stream`).

- A **Notebook Activity** was placed onto the pipeline canvas, pointing directly to our streaming notebook.
- Because the notebook utilizes an infinite loop (`while True`), this pipeline activity remains permanently in an *In progress* state, which is exactly the intended behavior for a live background ingestion engine.

Step 3: Eventstream & KQL Eventhouse

The Eventstream (`pl_iss_tracker_stream_es`) acts as the highly scalable ingest gateway for the data platform. The transmitted JSON payloads are captured here instantly. From the Eventstream, a direct data connection was routed to a **KQL Database** (`db_iss_tracker`) inside an Eventhouse.

During ingestion, Fabric automatically parses the JSON stream, structuring it directly into the `Live_iss_data` table with the designated columns: `timestamp`, `latitude`, `longitude`, and `craft`.

Step 4: Automated Data Retention Policy (KQL)

The ISS takes exactly **92 minutes** to complete a full orbit around the Earth. Since historical data was not required for this real-time tracking use case, and to prevent the map visual from turning into an unreadable mess of overlapping lines, a proactive retention policy was established at the database layer.

The following administrative command was executed within the KQL Queryset to enforce this logic:

```
.alter-merge table Live_iss_data policy retention softdelete = 92m recoverability = disabled
```

The database engine now autonomously reviews rows on the fly, dropping data points that pass the 92-minute mark. This ensures that the storage contains an active 'moving window' representing exactly one full orbit.

Note: To start with a clean slate and clear old test points or connectivity gaps caused during initial setup, the table data can be completely flushed using: `.clear table Live_iss_data data`.

Step 5: Power BI Live Dashboard & Change Detection

The dashboard is directly connected to the KQL database utilizing a real-time **DirectQuery** connection.

- **Azure Maps Visual:** Live latitude and longitude columns map out the current flight path as a smooth connected line.
- **Auto-Centering:** Under the visual formatting properties of the map, **Auto zoom** was turned **On**. Because the underlying KQL data moves as a 92-minute bubble, the viewport automatically shifts and focuses on the active position of the ISS.
- **Admin Override via Change Detection:** The workspace administrator enforced a strict 5-minute minimum interval on standard *Auto page refresh*. To bypass this restriction, **Change Detection** was

enabled on the `timestamp` column. Power BI now continuously polls KQL with lightweight checks, immediately forcing a map redraw the exact millisecond a new coordinate arrives.

3. Complete Source Code (Fabric Notebook)

Below is the complete, optimized Python source code executed within the Microsoft Fabric Notebook to drive the real-time data stream:

```

# =====
# SETTINGS (FULLY UP-TO-DATE WITH YOUR LIVE SAS CONNECTION STRING)
# =====
# Gained connection string, with ';EntityPath=...' cleanly truncated from the end:
CONNECTION_STR = "Endpoint=sb://
esehdbglvb5sjwodtz4hzz.servicebus.windows.net/;SharedAccessKeyName=key_c77791ec-e5c3-40d4-
ba20-80945d876bae;SharedAccessKey=wco5zequTUQvrMpv0fFAynAFU97n5AavD+AEhMvM0og="

# Official Event Hub Name mapping to your ingestion node:
EVENTHUB_NAME = "esehdbglvb5sjwodtz4hzz_eh"
# =====

import time
import requests
import json

# Libraries are resolved via the attached custom Fabric Environment
from azure.eventhub import EventHubProducerClient, EventData

API_URL = "http://api.open-notify.org/iss-now.json"

def get_iss_location():
    """Fetches live telemetry data from the public ISS REST API"""
    try:
        response = requests.get(API_URL, timeout=10)
        if response.status_code == 200:
            data = response.json()
            return {
                "timestamp": int(data["timestamp"]),
                "latitude": float(data["iss_position"]["latitude"]),
                "longitude": float(data["iss_position"]["longitude"]),
                "craft": "Iss"
            }
    except Exception as e:
        print(f"⚠️ API Warning (automatically retrying): {e}")
    return None

# Start the continuous streaming engine loop
try:
    producer = EventHubProducerClient.from_connection_string(
        conn_str=CONNECTION_STR,
        eventhub_name=EVENTHUB_NAME
    )

    print("🚀 Initiating PERMANENT ISS data stream to Fabric Eventstream...")
    with producer:
        counter = 1
        while True:
            iss_data = get_iss_location()

            if iss_data:
                event_data_batch = producer.create_batch()
                event_data_batch.add(EventData(json.dumps(iss_data)))

```

```

        producer.send_batch(event_data_batch)
        print(f"📡 [{counter}] Ingested to Eventstream -> Lat:
{iss_data['latitude']} | Lon: {iss_data['longitude']}")
        counter += 1
    else:
        print("⌚ Unable to reach endpoint, retrying in next cycle...")

    time.sleep(5)
except Exception as e:
    print(f"❌ Critical error in stream-engine runtime: {e}")

```

Status: The full end-to-end telemetry pipeline is fully operational. The database handles autonomous housekeeping, the pipeline activity provides persistent scheduling, and the Power BI layout provides an active, lag-free mission control environment.